

High-Performance Computing Systems - Lab 1

Performance Analysis and Optimization of the Sobel Algorithm

Angelos Gkertsos (03658) Nikolaos Kalamaris (03768)

ECE4415 - October 2025

Contents

1	System Specifications	3
1.1	Apple macOS System	3
1.2	Linux System	3
2	Introduction	3
3	Optimization Steps and Results	4
3.1	Initial Implementation (sobel_orig.c)	4
3.2	Loop Fusion (sobel_convolution2D_1.c)	4
3.3	Common Subexpression Elimination (sobel_convolution2D_2.c)	4
3.4	Loop Unrolling & Constant Indexing (sobel_convolution2D_3.c)	5
3.5	Mathematical Function Call Elimination (sobel_p_pow_removal.c)	5
3.6	Lookup Table Optimization (sobel_lookup_table.c)	5
3.7	Loop Unrolling (sobel_loop_unroll_16pixels.c)	6
3.8	Manual Function Inlining (sobel_16pixels_func_inling.c)	6
3.9	Mathematical Function Call Elimination in PSNR (sobel_psnr_pow_removal.c)	7
3.10	Loop Fusion in PSNR (sobel_psnr_loop_fusion.c)	7
3.11	Redundant Calculation Removal (sobel_operator_0s_calc_removal.c)	7
3.12	Redundant Calculation Removal (sobel_operator_1s_calc_removal.c)	8
3.13	Constant Propagation (sobel_operator_removal.c)	8
3.14	Pointer Arithmetic Optimization (sobel_input_pointers.c)	8
3.15	Guarded Calculation (sobel_if_diff.c)	9
3.16	Common Subexpression Elimination & Strength Reduction (sobel_cur_row_expression_1.c)	9
3.17	Common Subexpression Elimination Refinement (sobel_cur_row_expression_2.c)	9
3.18	Common Subexpression Elimination for Current Pixel (sobel_cur_pixel_expression.c)	10
3.19	Complete Strength Reduction (sobel_mult_remove.c)	10
3.20	Sliding Window Optimization (sobel_less_ptr_output.c)	10
3.21	Pointer Restriction Optimization (sobel_restricts.c)	11
3.22	Algorithmic Optimization (Sliding Column Sums) (sobel_columns_sums.c)	11
3.23	Register Allocation Hint (sobel_columns_sums_registers_FINAL_VERSION.c)	12
4	Aggregate Analysis	13
4.0.1	The Complete Optimization Journey	13
4.0.2	Sequential Improvement Analysis	14
4.0.3	Final Performance Ranking	17
4.0.4	Impact of Compiler Optimization (-O3 vs -O0)	20
5	Discussion of Failed Optimizations	22
6	Conclusion	22

1 System Specifications

The experiments for this project were conducted on two distinct machines to ensure a comprehensive analysis. The hardware and software specifications for each system are detailed below.

1.1 Apple macOS System

- **Model:** MacBook Pro
- **Processor:** Apple M3 Pro
- **Memory:** 18 GB
- **Operating System:** macOS Sequoia 15.0.1 (Build 24A348)
- **Kernel Version:** 24.0.0
- **Compiler:** Apple clang version 15.0.0 (clang-1500.3.9.4)

1.2 Linux System

- **Processor:** AMD Ryzen 5 5600H with Radeon Graphics (6 cores / 12 threads)
- **Memory:** 5.6 GiB
- **Operating System:** Ubuntu 24.04.2 LTS
- **Kernel Version:** 6.8.0-48-generic
- **Compiler:** Intel oneAPI DPC++/C++ Compiler 2025.2.1 (icx)

2 Introduction

The Sobel operator is a fundamental algorithm in image processing used for edge detection. The primary objective of this project is to systematically optimize a baseline C implementation of the Sobel algorithm. Starting with the unoptimized version, we will apply a series of incremental code modifications. The performance of each version will be rigorously measured and analyzed, both with and without compiler optimizations enabled (`-O0` vs `-Ofast`). This report documents the optimization journey, evaluates the impact of each change, and presents a comprehensive analysis of the results to identify the most effective optimization strategies.

3 Optimization Steps and Results

The optimization process was iterative, with each new version building upon the performance gains of its predecessor. Each modification is detailed below, along with its technical rationale and a brief analysis of its impact. **It is also crucial to note a shift in our optimization strategy.** After step 3.16 (`sobel_cur_row_expression_1.c`), our primary criterion for accepting a new optimization became its performance impact on the **-O3 build**. The goal was to achieve the best possible fully-optimized time. The only exception was if a change introduced a severe performance degradation on the -O0 build while offering only a negligible gain on the -O3 build.

While the following subsections detail each optimization step sequentially, the graphical chart for each new version will be presented in the Excel Spreadsheet, to reduce the length of the report.

3.1 Initial Implementation (`sobel_orig.c`)

Description: The baseline, unoptimized C implementation of the Sobel algorithm.

Rationale: This version serves as the performance reference point.

Analysis: This is the baseline.

Linux (Ryzen 5):

-O0: Mean: 1.7146s (StdDev: 0.0138s).

-O3: Mean: 0.2112s (StdDev: 0.0026s).

macOS (M3 Pro):

-O0: Mean: 0.9760s (StdDev: 0.0043s).

-O3: Mean: 0.1141s (StdDev: 0.0030s).

3.2 Loop Fusion (`sobel_convolution2D_1.c`)

Description: The nested loops from the `convolution2D()` function were merged directly into the main loops that iterate over the image pixels.

Rationale: Loop control overhead reduction and temporal data locality optimization.

Analysis:

Linux (Ryzen 5):

-O0: Change: +12.70% (Speedup: 1.15x). New Mean: 1.4968s (StdDev: 0.0014s).

-O3: Change: +3.11% (Speedup: 1.03x). New Mean: 0.2046s (StdDev: 0.0049s).

macOS (M3 Pro):

-O0: Change: +11.77% (Speedup: 1.13x). New Mean: 0.8612s (StdDev: 0.0160s).

-O3: Change: -1.17% (Speedup: 0.99x). New Mean: 0.1154s (StdDev: 0.0035s).

3.3 Common Subexpression Elimination (`sobel_convolution2D_2.c`)

Description: Before processing each pixel, the base memory addresses for the previous, current, and next rows were pre-calculated and stored in local variables. These variables were then reused for accessing pixel data.

Rationale: Avoid re-calculating `'i*cols + j'` multiple times.

Analysis:

Linux (Ryzen 5):

-O0: Change: -1.91% (Speedup: 0.98x). New Mean: 1.5254s (StdDev: 0.0100s).

-O3: Change: +0.48% (Speedup: 1.00x). New Mean: 0.2036s (StdDev: 0.0033s).

macOS (M3 Pro):

-O0: Change: -2.81% (Speedup: 0.97x). New Mean: 0.8854s (StdDev: 0.0154s).

-O3: Change: -1.41% (Speedup: 0.99x). New Mean: 0.1170s (StdDev: 0.0027s).

3.4 Loop Unrolling & Constant Indexing (sobel_convolution2D_3.c)

Description: The 'for' loop was completely unrolled. Use of constant indices to access the operator's values.

Rationale: We eliminate loop control overhead. Second, we use constant indices allows the compiler to calculate the exact memory addresses of kernel elements at compile time.

Analysis:

Linux (Ryzen 5):

-O0: Change: +1.63% (Speedup: 1.02x). New Mean: 1.5005s (StdDev: 0.0193s).

-O3: Change: +26.59% (Speedup: 1.36x). New Mean: 0.1495s (StdDev: 0.0021s).

macOS (M3 Pro):

-O0: Change: +7.15% (Speedup: 1.08x). New Mean: 0.8221s (StdDev: 0.0176s).

-O3: Change: +0.92% (Speedup: 1.01x). New Mean: 0.1160s (StdDev: 0.0031s).

3.5 Mathematical Function Call Elimination (sobel_p_pow_removal.c)

Description: The 'pow(variable, 2)' function calls was replaced with direct multiplication. ($p = (\text{gradient_x} * \text{gradient_x}) + (\text{gradient_y} * \text{gradient_y})$);

Rationale: This change eliminates the significant overhead of the function call and replaces a complex operation with a simple multiplication.

Analysis: This was one of the most critical optimizations. The multiplication instruction provided a massive speedup on both systems, especially for the -O0 builds.

Linux (Ryzen 5):

-O0: Change: +46.87% (Speedup: 1.88x). New Mean: 0.7972s (StdDev: 0.0123s).

-O3: Change: +8.18% (Speedup: 1.09x). New Mean: 0.1373s (StdDev: 0.0015s).

macOS (M3 Pro):

-O0: Change: +51.09% (Speedup: 2.04x). New Mean: 0.4021s (StdDev: 0.0058s).

-O3: Change: +2.92% (Speedup: 1.03x). New Mean: 0.1126s (StdDev: 0.0033s).

3.6 Lookup Table Optimization (sobel_lookup_table.c)

Description: An array, sq_lookup_table[], was created to act as a lookup table of the squared values of all possible gradient sums.

Rationale: Since pixel values are in a known range [0, 255] (grayscale images), their squares can be pre-calculated and stored in an array instead of performing a multiplication operation at runtime for every pixel.

Analysis: A potential architectural difference. The Lookup Table trades a multiplication for a memory access. The Linux (Ryzen) system benefited from this. However, the macOS (M3 Pro) -O3 build became slower, which strongly suggests that the M3 Pro's multiplication instruction is significantly faster than its cache access, or that the LUT polluted the cache.

Linux (Ryzen 5):

-O0: Change: +9.06% (Speedup: 1.10x). New Mean: 0.7250s (StdDev: 0.0047s).

-O3: Change: +14.13% (Speedup: 1.16x). New Mean: 0.1179s (StdDev: 0.0010s).

macOS (M3 Pro):

-O0: Change: +11.04% (Speedup: 1.12x). New Mean: 0.3577s (StdDev: 0.0006s).

-O3: Change: -2.55% (Speedup: 0.98x). New Mean: 0.1154s (StdDev: 0.0016s).

3.7 Loop Unrolling (sobel_loop_unroll_16pixels.c)

Description: The main processing loop, which iterates over the columns of the image, was unrolled by a factor of 16, processing 16 pixels in one iteration. A cleanup loop was added to handle the remaining edge pixels.

Rationale: This optimization reduces loop overhead. For every 16 pixels, it executes only one loop counter increment, one comparison, and one branch instruction, instead of 16 of each.

Analysis: This optimization was a significant failure, especially on the -O3 builds. Unrolling the loop by 16x likely polluted the instruction cache (forcing the CPU to fetch new instructions constantly) and created a loop body too large for the compiler to optimize effectively, forcing it to spill variables from registers back to memory (register spilling).

Linux (Ryzen 5):

-O0: Change: -4.94% (Speedup: 0.95x). New Mean: 0.7608s (StdDev: 0.0016s).

-O3: Change: -102.77% (Speedup: 0.49x). New Mean: 0.2390s (StdDev: 0.0032s).

macOS (M3 Pro):

-O0: Change: -0.93% (Speedup: 0.99x). New Mean: 0.3610s (StdDev: 0.0004s).

-O3: Change: -15.76% (Speedup: 0.86x). New Mean: 0.1336s (StdDev: 0.0040s).

3.8 Manual Function Inlining (sobel_16pixels_func_inling.c)

Description: The call to the ‘Convolution_2D’ function was removed, the logic was manually placed directly inside the main processing loop for each pixel.

Rationale: Elimination of overhead of a function call for every pixel. The execution flow becomes linear and significantly faster, especially for a function called millions of times.

Analysis: This step fixed the critical flaw from the previous version. The loop unrolling only unrolled the function calls, not the logic itself. This version removed the function call overhead and the compiler could now see the full instruction stream.

Linux (Ryzen 5):

-O0: Change: +34.47% (Speedup: 1.53x). New Mean: 0.4986s (StdDev: 0.0075s).

-O3: Change: +56.87% (Speedup: 2.32x). New Mean: 0.1031s (StdDev: 0.0005s).

macOS (M3 Pro):

-O0: Change: +27.64% (Speedup: 1.38x). New Mean: 0.2612s (StdDev: 0.0014s).

-O3: Change: +36.85% (Speedup: 1.58x). New Mean: 0.0844s (StdDev: 0.0023s).

3.9 Mathematical Function Call Elimination in PSNR (sobel_psnr_pow_removal.c)

Description: The ‘pow(diff, 2)’ function call inside PSNR calculation was replaced with direct multiplication ‘diff * diff’.

Rationale: This change eliminates the significant overhead of the function call and replaces a complex operation with a simple multiplication.

Analysis:

Linux (Ryzen 5):

-O0: Change: +17.75% (Speedup: 1.22x). New Mean: 0.4101s (StdDev: 0.0023s).

-O3: Change: -1.55% (Speedup: 0.98x). New Mean: 0.1047s (StdDev: 0.0003s).

macOS (M3 Pro):

-O0: Change: +4.43% (Speedup: 1.05x). New Mean: 0.2497s (StdDev: 0.0002s).

-O3: Change: +0.72% (Speedup: 1.01x). New Mean: 0.0838s (StdDev: 0.0014s).

3.10 Loop Fusion in PSNR (sobel_psnr_loop_fusion.c)

Description: The loops inside the ‘psnr()’ function were merged into the main single loop, calculating the PSNR at the end of every pixel’s process.

Rationale: Fusing these loops improves data locality and reduces loop control overhead for the final stage of the program.

Analysis: This optimization shows a compiler difference. Fusing the loops made the -O0 significantly slower. This is possible due to increased register pressure. The loop needs to track multiple variables simultaneously, forcing -O0 compiler to spill to the stack. The -O3 compiler can manage this so it benefits from the reduced loop overhead.

Linux (Ryzen 5):

-O0: Change: -34.87% (Speedup: 0.74x). New Mean: 0.5531s (StdDev: 0.0004s).

-O3: Change: +11.43% (Speedup: 1.13x). New Mean: 0.0927s (StdDev: 0.0002s).

macOS (M3 Pro):

-O0: Change: -6.10% (Speedup: 0.94x). New Mean: 0.2649s (StdDev: 0.0001s).

-O3: Change: +10.23% (Speedup: 1.11x). New Mean: 0.0752s (StdDev: 0.0013s).

3.11 Redundant Calculation Removal (sobel_operator_0s_calc_removal.c)

Description: Eliminated parts of the Sobel calculation that involved multiplication by zero.

Rationale: This is a form of dead code elimination.

Analysis:

Linux (Ryzen 5):

-O0: Change: +27.44% (Speedup: 1.38x). New Mean: 0.4013s (StdDev: 0.0004s).

-O3: Change: +23.15% (Speedup: 1.30x). New Mean: 0.0713s (StdDev: 0.0005s).

macOS (M3 Pro):

-O0: Change: +22.20% (Speedup: 1.29x). New Mean: 0.2061s (StdDev: 0.0001s).

-O3: Change: +31.89% (Speedup: 1.47x). New Mean: 0.0512s (StdDev: 0.0005s).

3.12 Redundant Calculation Removal (sobel_operator_1s_calc_removal.c)

Description: Simplified calculations involving multiplication by 1.

Rationale: This is an algebraic simplification. Multiplication by ‘1’ is a redundant operation.

Analysis:

Linux (Ryzen 5):

-O0: Change: +1.45% (Speedup: 1.01x). New Mean: 0.3955s (StdDev: 0.0006s).

-O3: Change: +13.65% (Speedup: 1.16x). New Mean: 0.0615s (StdDev: 0.0005s).

macOS (M3 Pro):

-O0: Change: +2.58% (Speedup: 1.03x). New Mean: 0.2008s (StdDev: 0.0002s).

-O3: Change: +24.78% (Speedup: 1.33x). New Mean: 0.0385s (StdDev: 0.0001s).

3.13 Constant Propagation (sobel_operator_removal.c)

Description: Replaced Sobel operators with their known constant values

Rationale: This change removes memory access operations for the kernel.

Analysis:

Linux (Ryzen 5):

-O0: Change: +8.72% (Speedup: 1.10x). New Mean: 0.3610s (StdDev: 0.0007s).

-O3: Change: +27.04% (Speedup: 1.37x). New Mean: 0.0449s (StdDev: 0.0003s).

macOS (M3 Pro):

-O0: Change: +3.84% (Speedup: 1.04x). New Mean: 0.1931s (StdDev: 0.0001s).

-O3: Change: +34.04% (Speedup: 1.52x). New Mean: 0.0254s (StdDev: 0.0001s).

3.14 Pointer Arithmetic Optimization (sobel_input_pointers.c)

Description: Instead of calculating an index for the ‘input’ array in each step, three pointers now traverse the image rows. These pointers are directly dereferenced to access pixel data.

Rationale: This is a form of strength reduction. It replaces the multiplication and addition operations required for array index calculation with increment operations.

Analysis:

Linux (Ryzen 5):

-O0: Change: +19.43% (Speedup: 1.24x). New Mean: 0.2909s (StdDev: 0.0009s).

-O3: Change: -0.08% (Speedup: 1.00x). New Mean: 0.0449s (StdDev: 0.0003s).

macOS (M3 Pro):

-O0: Change: +7.11% (Speedup: 1.08x). New Mean: 0.1793s (StdDev: 0.0001s).

-O3: Change: -0.07% (Speedup: 1.00x). New Mean: 0.0254s (StdDev: 0.0000s).

3.15 Guarded Calculation (sobel_if_diff.c)

Description: The full calculation for the PSNR is now only performed for each pixel if its 'diff' is not 0 (not identical pixels). This is an optimization only for our script since computed Sobel image should always be infinite.

Rationale: This is a guarded calculation optimization. This eliminates the additions of zeroes to the total PSNR.

Analysis:

Linux (Ryzen 5):

-O0: Change: +11.24% (Speedup: 1.13x). New Mean: 0.2582s (StdDev: 0.0008s).

-O3: Change: +7.70% (Speedup: 1.08x). New Mean: 0.0415s (StdDev: 0.0006s).

macOS (M3 Pro):

-O0: Change: +18.31% (Speedup: 1.22x). New Mean: 0.1465s (StdDev: 0.0001s).

-O3: Change: +15.53% (Speedup: 1.18x). New Mean: 0.0215s (StdDev: 0.0001s).

3.16 Common Subexpression Elimination & Strength Reduction (sobel_cur_row_expression_1.c)

Description: A bit-shift operation by 1 was also used to replace multiplication by 2. New variable cur_row holds the value of the current's pixel row and is used to avoid multiplications to calculate the values of row_0 and row_2 with as simple addition and subtraction.

Rationale: Faster bit-shift instruction instead of multiplication. Common Subexpression Elimination avoids re-calculating the row sums.

Analysis:

Linux (Ryzen 5):

-O0: Change: -0.23% (Speedup: 1.00x). New Mean: 0.2588s (StdDev: 0.0006s).

-O3: Change: +1.86% (Speedup: 1.02x). New Mean: 0.0407s (StdDev: 0.0006s).

macOS (M3 Pro):

-O0: Change: -0.07% (Speedup: 1.00x). New Mean: 0.1466s (StdDev: 0.0001s).

-O3: Change: -0.04% (Speedup: 1.00x). New Mean: 0.0215s (StdDev: 0.0000s).

3.17 Common Subexpression Elimination Refinement (sobel_cur_row_expression_2.c)

Description: The cur_row variable replaces every 'j * SIZE' expression.

Rationale: Storing and reusing the result reduces the total number of arithmetic operations.

Analysis: This optimization shows a clear performance divergence. On the Linux (Ryzen 5) system, it resulted in a notable -4.20% performance degradation for the -O0 build and this version would have been discarded. However, we proceeded based on the macOS (M3 Pro) results, where the change was negligible (Speedup: 0.99x at -O0). This version was kept to demonstrate that Common Subexpression Elimination is in theory a valid optimization and should perform better on different machines.

Linux (Ryzen 5):

-O0: Change: -4.20% (Speedup: 0.96x). New Mean: 0.2696s (StdDev: 0.0008s).

-O3: Change: -0.71% (Speedup: 0.99x). New Mean: 0.0410s (StdDev: 0.0010s).

macOS (M3 Pro):

-O0: Change: -0.56% (Speedup: 0.99x). New Mean: 0.1474s (StdDev: 0.0001s).
-O3: Change: -0.05% (Speedup: 1.00x). New Mean: 0.0215s (StdDev: 0.0000s).

3.18 Common Subexpression Elimination for Current Pixel (sobel_cur_pixel_expression.c)

Description: A new variable, ‘cur_pixel’, was introduced to pre-calculate and store the absolute index of the current pixel.

Rationale: This change eliminates redundant index calculations inside the main loop.

Analysis: This optimization on the other hand was ultimately discarded. While it provided a minor 3.10% improvement on the Linux -O0 build, it resulted in a performance degradation on both the macOS -O0 build (-2.64%) and the Linux -O3 build (-1.13%). This suggests that adding another variable increased register pressure for the -O0 compilers.

Linux (Ryzen 5):

-O0: Change: +3.10% (Speedup: 1.03x). New Mean: 0.2613s (StdDev: 0.0006s).

-O3: Change: -1.13% (Speedup: 0.99x). New Mean: 0.0415s (StdDev: 0.0008s).

macOS (M3 Pro):

-O0: Change: -2.64% (Speedup: 0.97x). New Mean: 0.1513s (StdDev: 0.0001s).

-O3: Change: -0.09% (Speedup: 1.00x). New Mean: 0.0215s (StdDev: 0.0000s).

3.19 Complete Strength Reduction (sobel_mult_remove.c)

Description: The multiplication by ‘-2’ was replaced with a bit-shift left by 1, and multiplications ‘-1’ were replaced with subtraction.

Rationale: This Strength Reduction optimization replaces every multiplication with a computationally cheaper equivalent.

Analysis: This optimization suggests a complex that the manual strength reduction confused the Intel (icx) compiler at -O0 leading it to generate less efficient machine code than it did for the original multiplications (thus the x0.90 speedup). The -O3 builds and the macOS (clang) build show no significant change, as those compilers perform this optimization automatically.

Linux (Ryzen 5):

-O0: Change: -11.04% (Speedup: 0.90x). New Mean: 0.2901s (StdDev: 0.0014s).

-O3: Change: +0.85% (Speedup: 1.01x). New Mean: 0.0411s (StdDev: 0.0010s).

macOS (M3 Pro):

-O0: Change: +1.84% (Speedup: 1.02x). New Mean: 0.1485s (StdDev: 0.0002s).

-O3: Change: +0.07% (Speedup: 1.00x). New Mean: 0.0215s (StdDev: 0.0000s).

3.20 Sliding Window Optimization (sobel_less_ptr_output.c)

Description: Temporary variables were introduced to store pixel values from the current window, implementing a sliding window approach to reduce memory accesses.

Rationale: Using the Sliding Window, instead of re-reading 9 pixels from memory for every single pixel, the code now reuses 6 of those values from the previous pixel’s calculation (which are held in temporary variables). After each pixel is processed, these variables are updated to “slide” the window: values from the right column become the new middle column, and values from the middle column become the new left column. The new pixel

process only needs to read 3 new pixels from the right-most column, drastically reducing memory accesses.

Analysis: Highly effective optimization on -O3 builds, as the compiler could keep the temporary variables in fast registers. However, it made the macOS -O0 build slower suggesting that -O0 clang compiler could not handle the increased register pressure from the new variables and was forced to spill them to the stack, making it slower than simply re-reading from memory.

Linux (Ryzen 5):

-O0: Change: +21.51% (Speedup: 1.27x). New Mean: 0.2277s (StdDev: 0.0011s).

-O3: Change: +19.82% (Speedup: 1.25x). New Mean: 0.0330s (StdDev: 0.0002s).

macOS (M3 Pro):

-O0: Change: -9.14% (Speedup: 0.92x). New Mean: 0.1621s (StdDev: 0.0009s).

-O3: Change: +29.93% (Speedup: 1.43x). New Mean: 0.0151s (StdDev: 0.0000s).

3.21 Pointer Restriction Optimization (sobel_restricts.c)

Description: The ‘restrict’ keyword was added to the function’s pointer arguments (‘input’, ‘output’, and ‘golden’).

Rationale: This keyword is a promise to the compiler that different pointers will not refer to overlapping memory regions (aliasing). This allows the compiler to perform more aggressive optimizations, such as reordering memory operations

Analysis: This optimization is a pure compiler hint. As expected, it had no effect on the -O0 builds.

Linux (Ryzen 5):

-O0: Change: +1.24% (Speedup: 1.01x). New Mean: 0.2249s (StdDev: 0.0011s).

-O3: Change: +8.93% (Speedup: 1.10x). New Mean: 0.0300s (StdDev: 0.0002s).

macOS (M3 Pro):

-O0: Change: -0.46% (Speedup: 1.00x). New Mean: 0.1628s (StdDev: 0.0001s).

-O3: Change: +3.25% (Speedup: 1.03x). New Mean: 0.0146s (StdDev: 0.0000s).

3.22 Algorithmic Optimization (Sliding Column Sums) (sobel_columns_sums.c)

Description: Introduced two temporary variables (‘old_right_column_sum_1’, ‘old_right_column_sum_2’) to store the sum of the right-most column for pixels ‘i’ and ‘i+1’, respectively.

Rationale: This is an advanced common subexpression Elimination using the unrolled loop. The right column sum of pixel ‘i’ is the additive inverse of the left column sum for pixel ‘i+2’. The two variables are necessary to hold the results from ‘i’ and ‘i+1’ simultaneously, as their reuse at ‘i+2’ and ‘i+3’ overlaps. This technique saves 3 memory reads and 2 additions per reused value.

Analysis: The macOS -O0 build again showed a slight degradation (-2.76%). This is again likely due to the increase of the two new temporary variables, forcing the -O0 clang compiler to spill registers to the stack.

Linux (Ryzen 5):

-O0: Change: +3.05% (Speedup: 1.03x). New Mean: 0.2180s (StdDev: 0.0011s).

-O3: Change: +2.33% (Speedup: 1.02x). New Mean: 0.0293s (StdDev: 0.0003s).

macOS (M3 Pro):

-O0: Change: -2.76% (Speedup: 0.97x). New Mean: 0.1673s (StdDev: 0.0001s).
-O3: Change: +5.09% (Speedup: 1.05x). New Mean: 0.0138s (StdDev: 0.0000s).

3.23 Register Allocation Hint (sobel_columns_sums_registers_FINAL_VERSION.c)

Description: The ‘register’ keyword was used to suggest that the compiler store frequently accessed variables in CPU registers.

Rationale: The ‘register’ keyword provides a direct hint to prioritize these specific variables. Storing them in registers minimizes memory access latency, providing a final small boost to the highly-optimized code.

Analysis::

Linux (Ryzen 5):

-O0: Change: -0.01% (Speedup: 1.00x). New Mean: 0.2181s (StdDev: 0.0009s).
-O3: Change: +0.95% (Speedup: 1.01x). New Mean: 0.0290s (StdDev: 0.0002s).

macOS (M3 Pro):

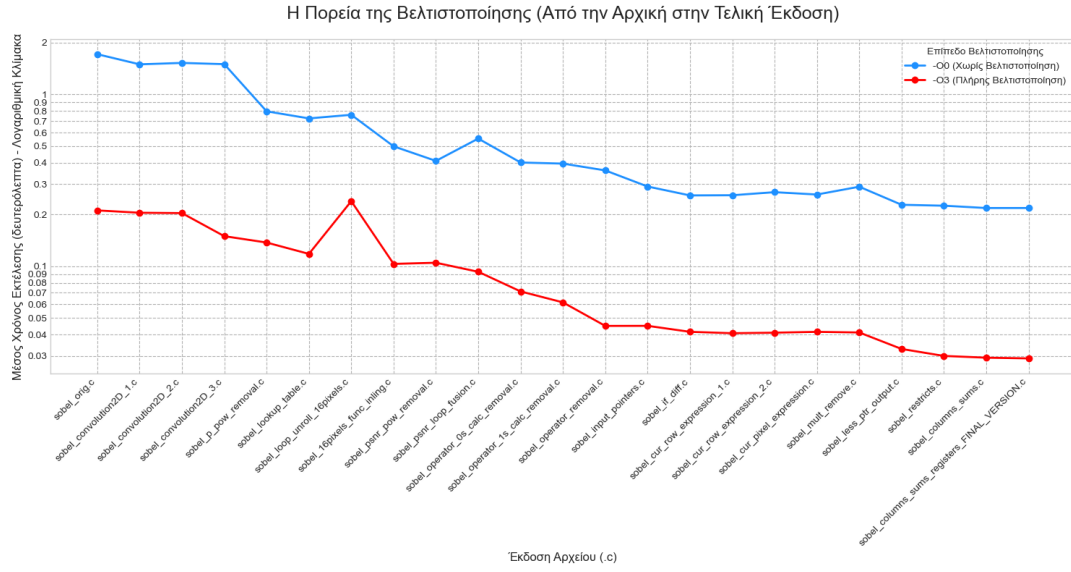
-O0: Change: +0.89% (Speedup: 1.01x). New Mean: 0.1659s (StdDev: 0.0022s).
-O3: Change: -0.11% (Speedup: 1.00x). New Mean: 0.0139s (StdDev: 0.0000s).

4 Aggregate Analysis

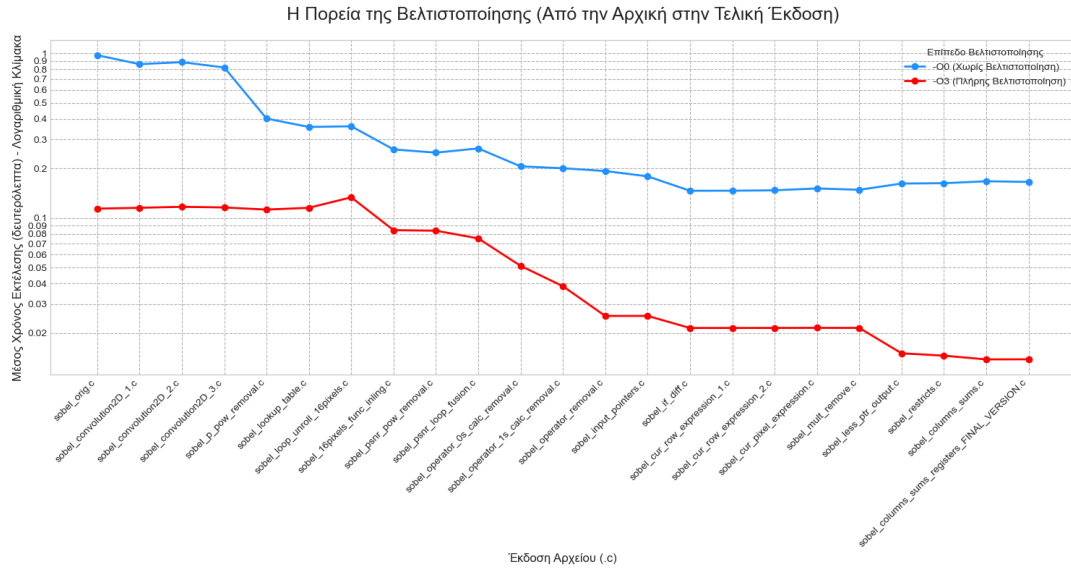
This section provides an overview of the optimization results.

4.0.1 The Complete Optimization Journey

The two figures below visualize the entire optimization process for the two different systems. The logarithmic y-axis clearly shows the progressive reduction in execution time with each step. The gap between the `-O0` and `-Ofast` lines demonstrates the consistent benefit of compiler optimizations, while the downward trend of both lines shows the effectiveness of our manual code changes.



(a) Optimization Journey (Linux - Ryzen 5)



(b) Optimization Journey (macOS - M3 Pro)

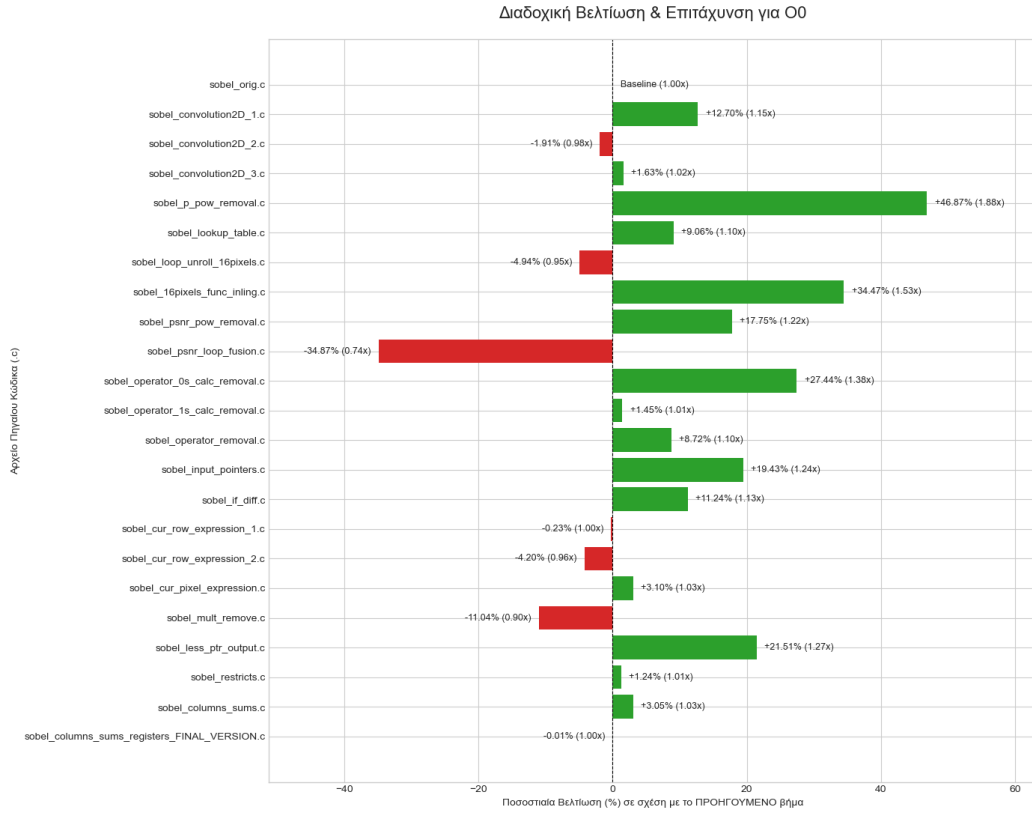
Key Observations from the Optimization Journey From the graphs in Figure 4.0.1, we can extract several key observations about the optimization process:

- **High-Impact, Low-Cost Wins:** The most effective optimization, especially for the -O0 builds was the removal of the `pow()` library function (`sobel_p_pow_removal.c`) replacing a function with a single multiplication instruction (over 2x speedup on the M3 Pro).
- **Optimization "Backfire":** The most dramatic performance degradation occurred at the `sobel_loop_unroll_16pixels.c` step. Unrolling the loop by 16 caused the pollution of the instruction cache and overwhelmed the compiler's optimizer, resulting in a 102% slowdown on the Linux -O3 build.
- **Correcting the Flaw:** The subsequent step, `sobel_16pixels_func_inling.c`, corrected this. The prior step had only unrolled the `convolution2D` function calls, not the logic. By inlining the convolution logic, the function call overhead was removed, resulting in one of the best optimizations (Speedup: 2.32x on Linux Ryzen 5)
- **Compiler and Architecture Divergence:** Several optimizations revealed some differences between the systems. The `sobel_lookup_table.c` step, for instance, benefited the Linux (Ryzen) system but slowed down the macOS (M3 Pro) -O3 build. This potentially suggests the M3 Pro's integer multiplication is faster than its cache access.
- **The Power of Algorithmic Changes:** In the last stages, the optimizations that provided the most consistent gains for the already-fast -O3 code were algorithmic, such as the "Sliding Window" (`sobel_less_ptr_output.c`) and "Sliding Column Sums" (`sobel_columns_sums.c`), which focused on reducing memory accesses rather than just arithmetic operations.

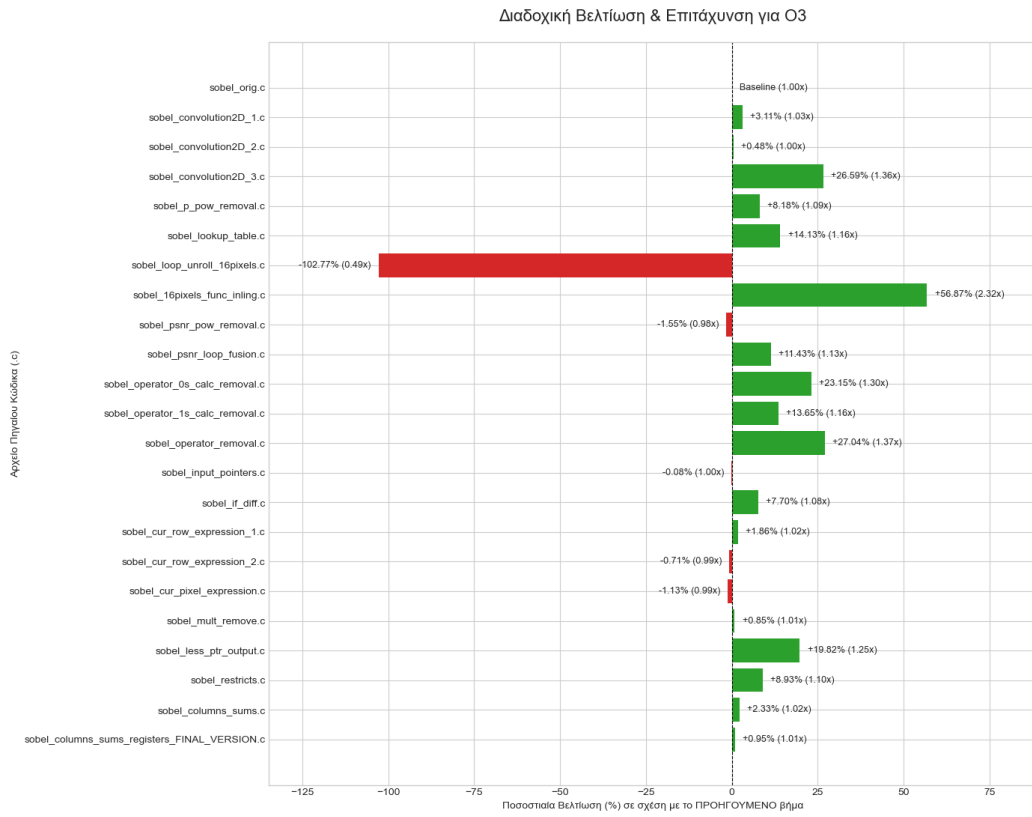
4.0.2 Sequential Improvement Analysis

The following figures detail the marginal benefit (improvement percentage and speedup factor) of each sequential optimization step. This analysis is presented for both test systems and both optimization levels (-O0 and -O3).

Figure 2: Sequential improvement analysis for the Linux (Ryzen 5) system.

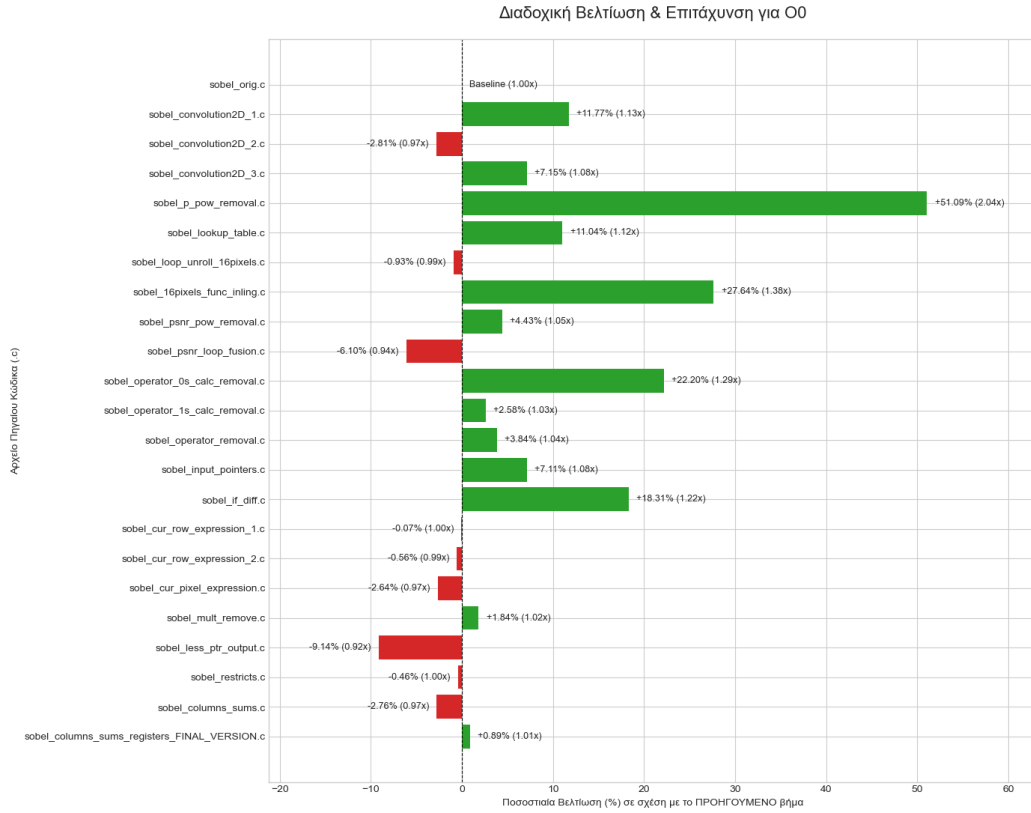


(a) Sequential Improvement (-O0) on Linux (Ryzen 5)

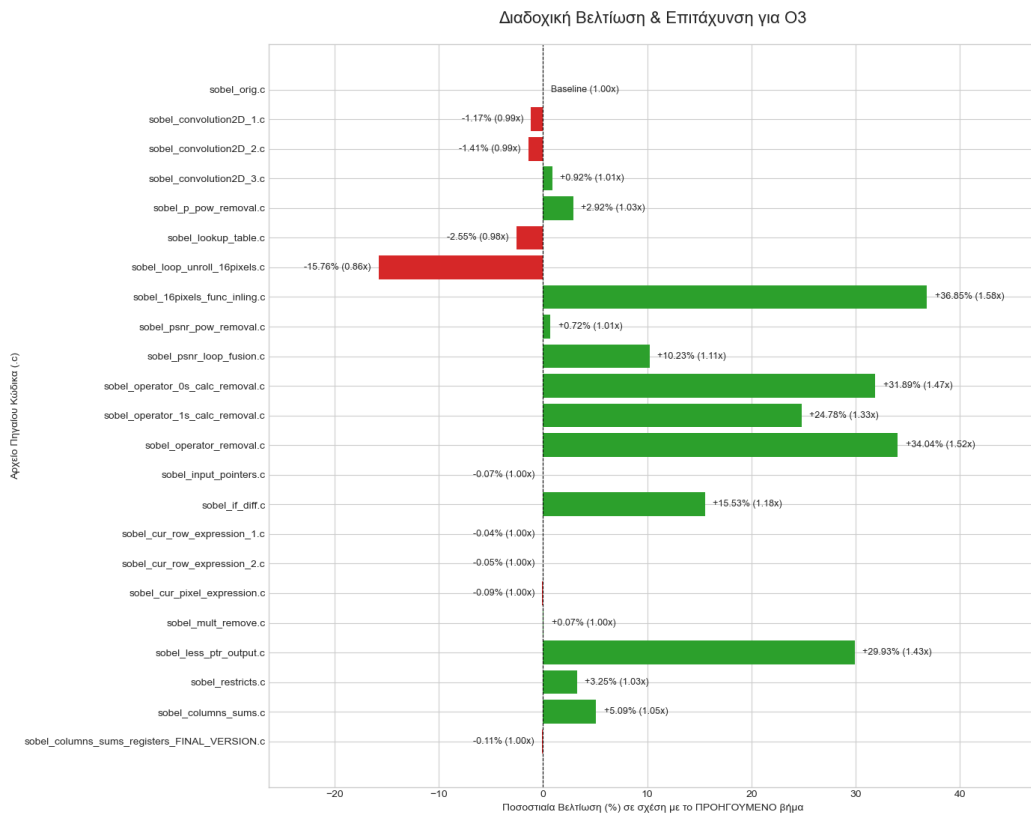


(b) Sequential Improvement (-O3) on Linux (Ryzen 5)

Figure 3: Sequential improvement analysis for the macOS (M3 Pro) system.



(a) Sequential Improvement (-O0) on macOS (M3 Pro)



(b) Sequential Improvement (-O3) on macOS (M3 Pro)

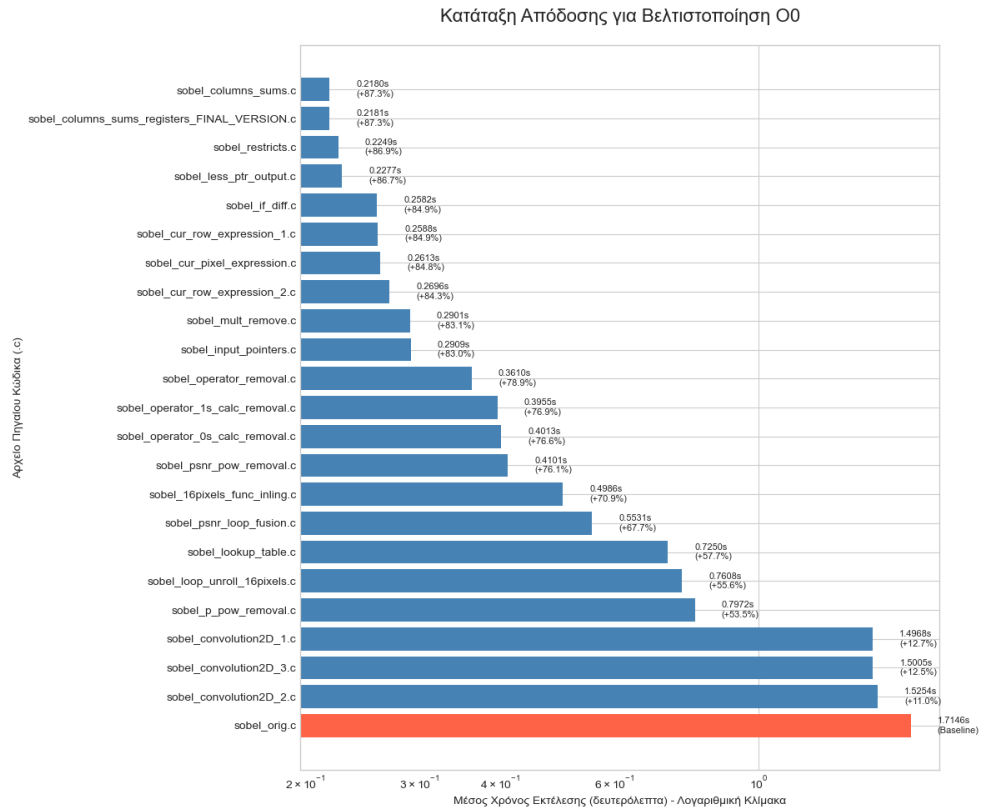
4.0.3 Final Performance Ranking

To assess the overall effectiveness of the optimizations, the final performance ranking for all code versions is presented below. These rankings are shown for both test systems and both optimization levels. **It is important to note that the final version may not rank highest in the -O0 builds, as our optimization strategy prioritized maximizing performance in the -O3 builds, even if some changes did not benefit (or slightly hindered) the unoptimized code.**

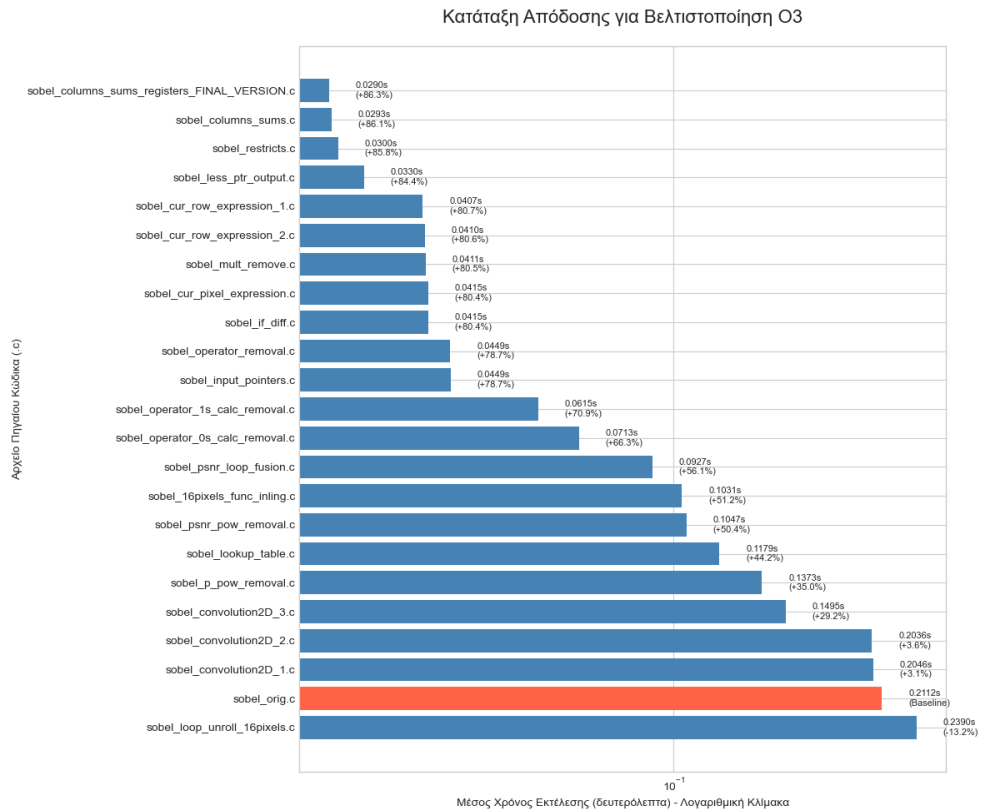
Here's a breakdown of why this occurs:

- **Shift in Focus:** As mentioned any optimization improving speed was accepted. Later, the primary goal became achieving the best performance with -O3 enabled. This became the main criterion for keeping changes, especially towards the end. Over several steps, these small, accepted -O0 slowdowns accumulated, potentially making the final version slightly slower at -O0.
- **Compiler vs. Manual Interaction:** Some manual optimizations happen when the compiler doesn't optimize (-O0) but might be redundant or interact poorly when the compiler optimizes aggressively (-O3). In our case, the last 8 versions show mostly benefits after the compiler optimization.
- **Increased Complexity & Register Pressure:** The biggest setback to the poor performance of (-O0) were optimizations (like `sobel_columns_sums.c`) introducing more temporary variables and complex logic. While an optimizing compiler (-O3) manages this well using registers, the (-O0) might struggle, forcing data into slower main memory ("register spilling").

Figure 4: Final performance ranking for the Linux (Ryzen 5) system.

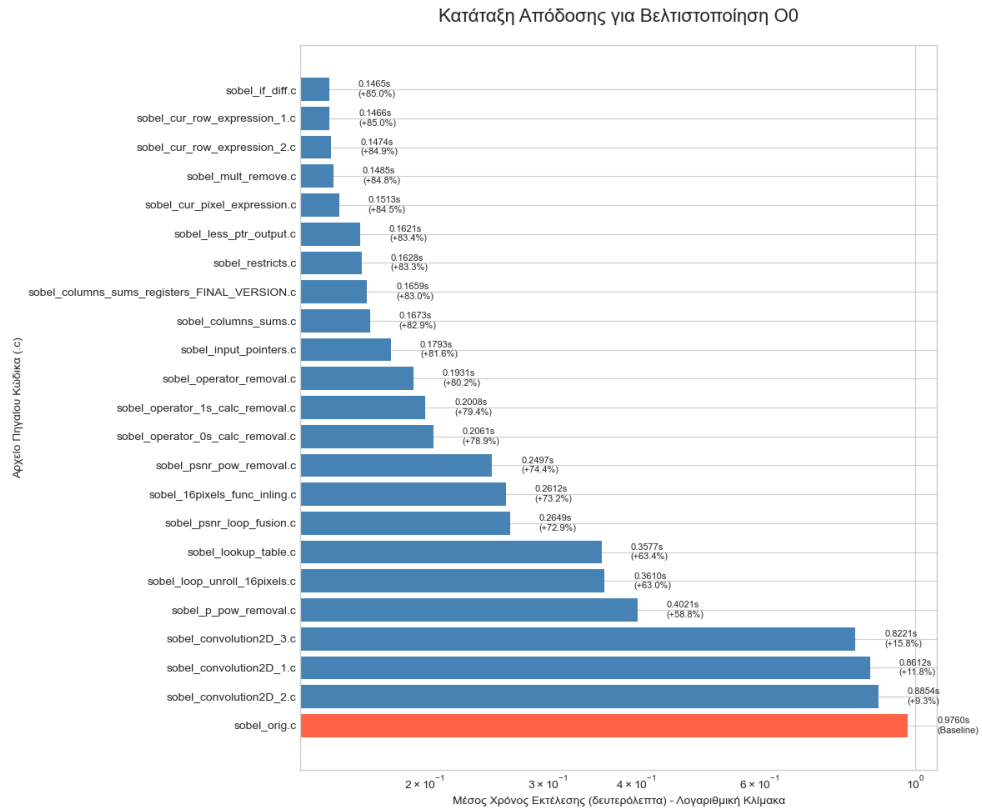


(a) Performance Ranking (-O0) on Linux (Ryzen 5)

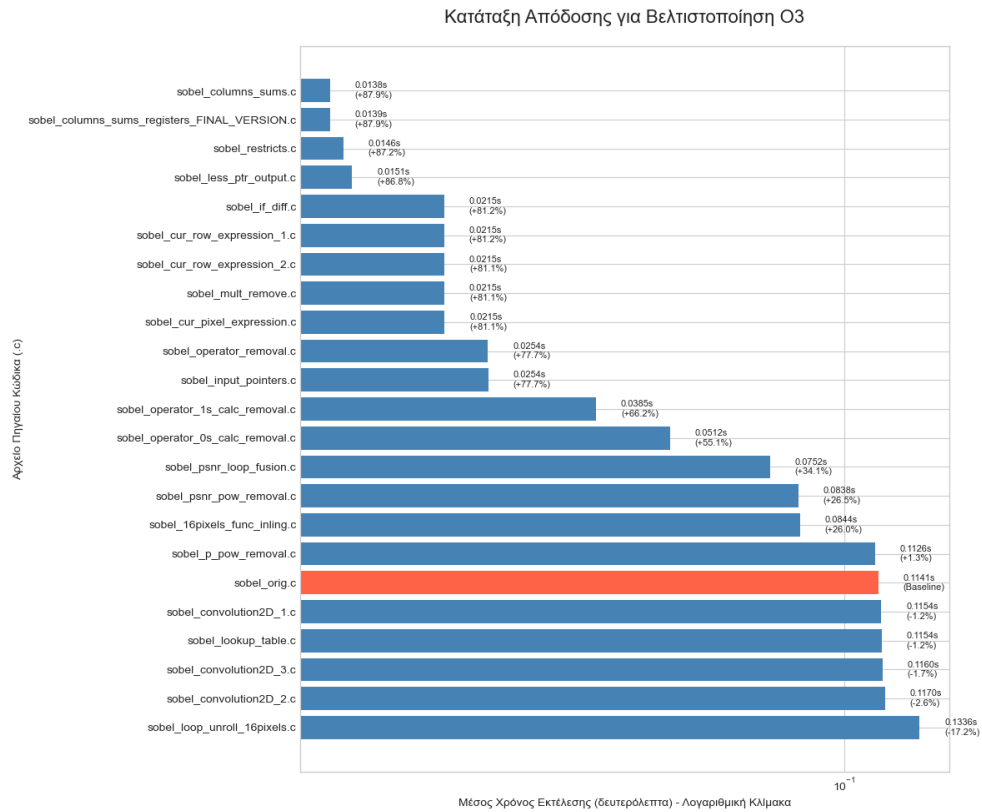


(b) Performance Ranking (-O3) on Linux (Ryzen 5)

Figure 5: Final performance ranking for the macOS (M3 Pro) system.



(a) Performance Ranking (-O0) on macOS (M3 Pro)



(b) Performance Ranking (-O3) on macOS (M3 Pro)

4.0.4 Impact of Compiler Optimization (-O3 vs -O0)

Following the previous analysis, to visualize the effectiveness of the compiler's optimizations relative to the unoptimized code, the following figures show the speedup factor achieved for each version on both systems. The speedup factor is calculated as (Execution Time at -O0) / (Execution Time at -O3). A higher bar indicates that the compiler was able to extract more performance on each version.

Figure 6: Compiler Speedup (-O3 vs -O0) on Linux (Ryzen 5). Labels show Speedup Factor (Percentage Improvement).

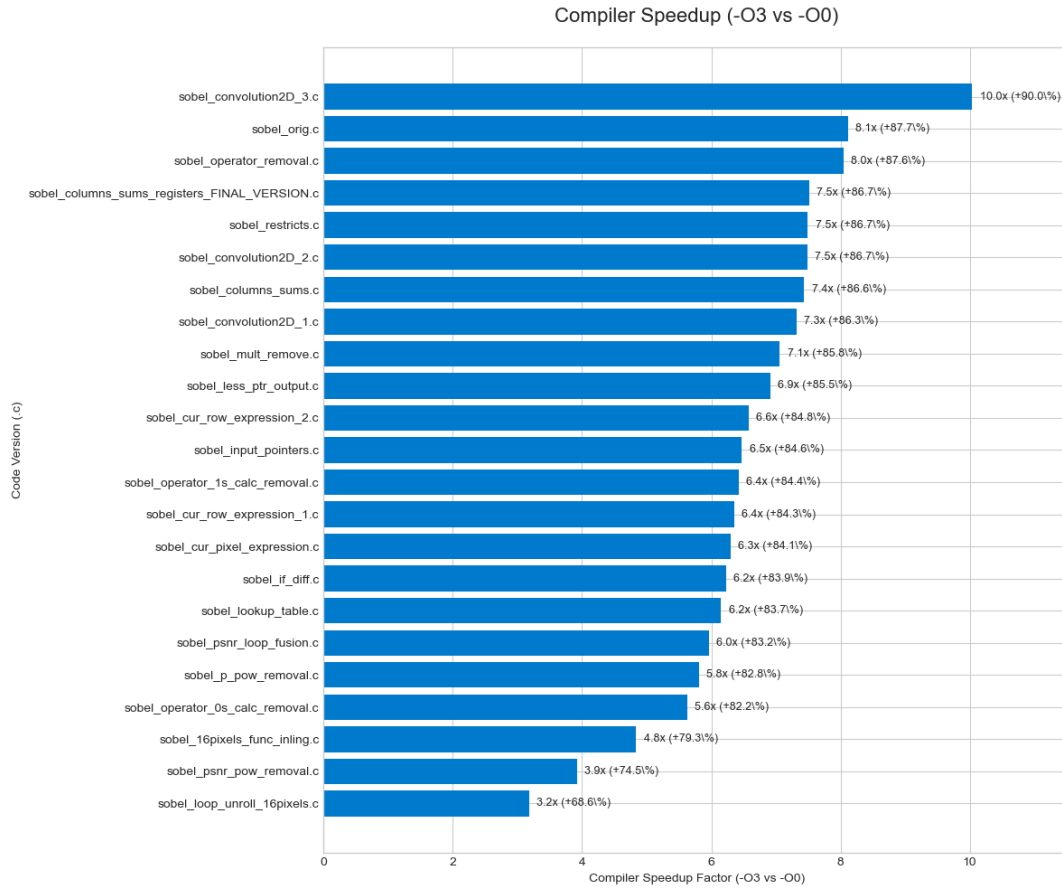
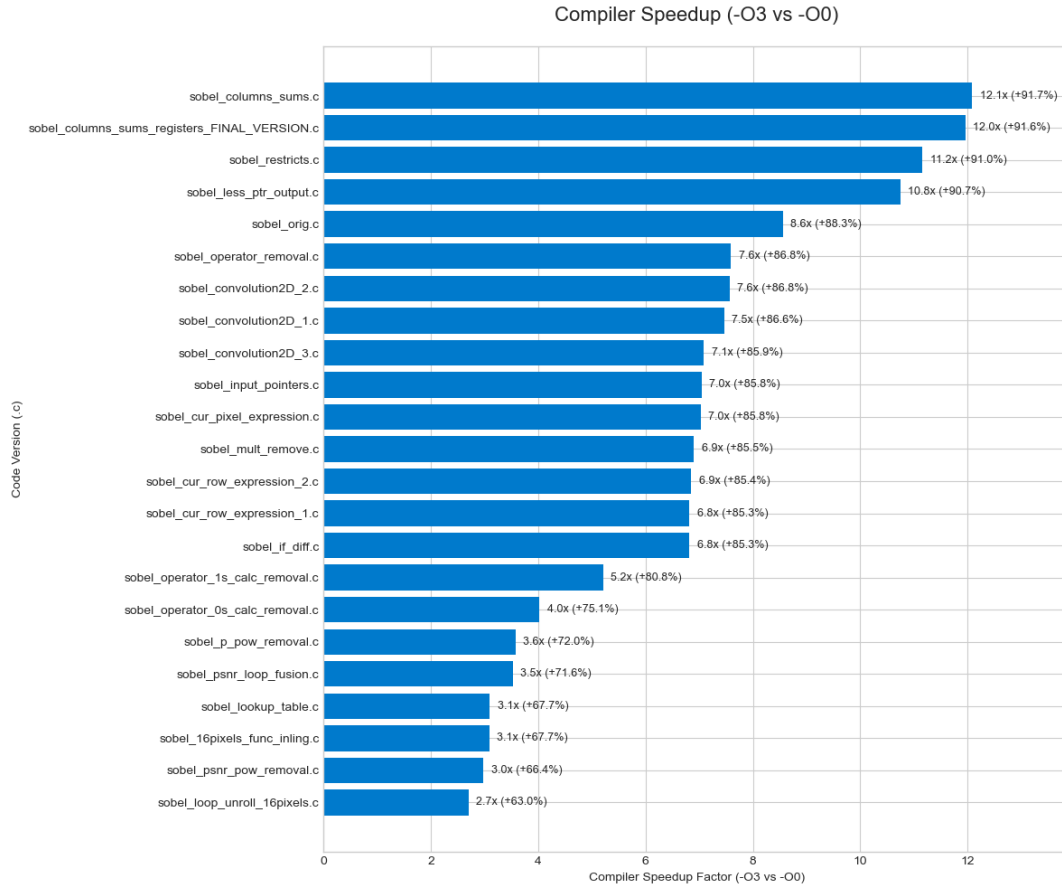


Figure 7: Compiler Speedup (-O3 vs -O0) on macOS (M3 Pro). Labels show Speedup Factor (Percentage Improvement).



From these plots (Figure 6 and 7), we observe that the final versions benefit substantially from compiler optimization, achieving high speedup factors. This confirms that our final code structure is optimized to support the compiler optimization techniques.

5 Discussion of Failed Optimizations

Not every attempted optimization yields positive results:

- **Aggressive Loop Unrolling (`sobel_loop_unroll_16pixels.c`):** . This version was kept only as a stepping stone to the next, corrective step, since it did not unroll the `convolution2D` calls. The next version (`sobel_16pixels_func_inling.c`) properly inlined this logic.
- **CSE for Current Pixel (`sobel_cur_pixel_expression.c`):** To eliminate redundant calculations the new variable `cur_pixel` had a performance degradation on the macOS -O0 build (-2.64%) and the Linux -O3 build (-1.13%). This suggests that the overhead of managing the extra variable outweighed the benefit of eliminating the index calculation. This version was discarded.
- **Strength Reduction Interaction (`sobel_mult_remove.c` on Linux -O0):** While replacing multiplications with bit-shifts and additions/subtractions is generally faster, this specific change caused an unexpected 11.04% slowdown on the Linux -O0 build. This highlights how manual micro-optimizations can sometimes confuse the compiler (like ‘-O0’). Since the -O3 builds were slightly improved, and the macOS -O0 build improved, this version was kept based on our strategy.

6 Conclusion

The sequential optimizations improved the performance of the Sobel algorithm. When compiled with -O3, the final version demonstrated a speedup of approximately **7.3x** on the Linux (Ryzen 5) system and **8.2x** on the macOS (M3 Pro) system compared to the respective original baseline versions. Similarly, when compiled without optimizations (-O0), the final version demonstrated a speedup of approximately **7.9x** on the Linux (Ryzen 5) system and **5.9x** on the macOS (M3 Pro) system compared to the respective original baseline versions.